NorteLeads

Segundo avance del proyecto de Bases de Datos II

Atención y seguimiento comercial para microempresas

Antoine Roman

COTECNOVA

Programa: Tecnologia en Gestión de Sistemas de Información

Asignatura: Bases de Datos II

Docente: Jhon James Cano Sánchez

Cartago, Valle del Cauca

25 de octubre de 2026

1 Descripción del proyecto

NorteLeads organiza la atención comercial de microempresas de Cartago y el norte del Valle del Cauca. La necesidad es conservar la información de las personas interesadas, asignar un responsable y registrar cada contacto. Cuando la información queda dispersa entre mensajes y notas, resulta difícil retomar una conversación o saber si una consulta llegó a una cita.

Los usuarios previstos son el administrador del servicio, los agentes que atienden interesados y un usuario de consulta. El administrador revisa y gestiona los datos; el agente registra seguimientos y cambios de estado; el usuario de consulta accede a información parcialmente oculta. Los datos de este avance son ficticios.

El alcance comprende diez tablas relacionadas, operaciones de lectura y seguimiento, historial de cambios y controles de acceso. La aplicación PHP comprueba la conexión y consulta una vista enmascarada. El envío real de mensajes, el inicio de sesión de agentes y la separación de información por negocio quedan para el proyecto final.

Cambios desde el primer avance

Se conservan las diez tablas y los tres procedimientos IN, OUT e INOUT. Se agregan validaciones con SIGNAL, dos funciones almacenadas, un trigger BEFORE INSERT y la cuenta de conexión al trigger de auditoría. También se incorporan dos vistas, tres roles y tres usuarios, hashes bcrypt para contraseñas de aplicación de prueba y cifrado del correo de los prospectos. La configuración de Docker se ajusta a los archivos presentes en el repositorio y se añade una migración con respaldo para conservar la base anterior.

Objetivos del proyecto

Organizar los prospectos por negocio; conservar el historial de contactos y citas; y permitir revisar el avance de las oportunidades mediante información consistente y accesos acordes con las responsabilidades de cada usuario.

2 Modelo de datos

El modelo mantiene diez tablas y once claves foráneas. La relación principal conecta negocios con prospectos y agentes; seguimientos y citas conservan operaciones concretas de cada persona. Los cambios de este corte afectan columnas y rutinas, no el número de tablas.

Figura 1

MER actualizado de NorteLeads

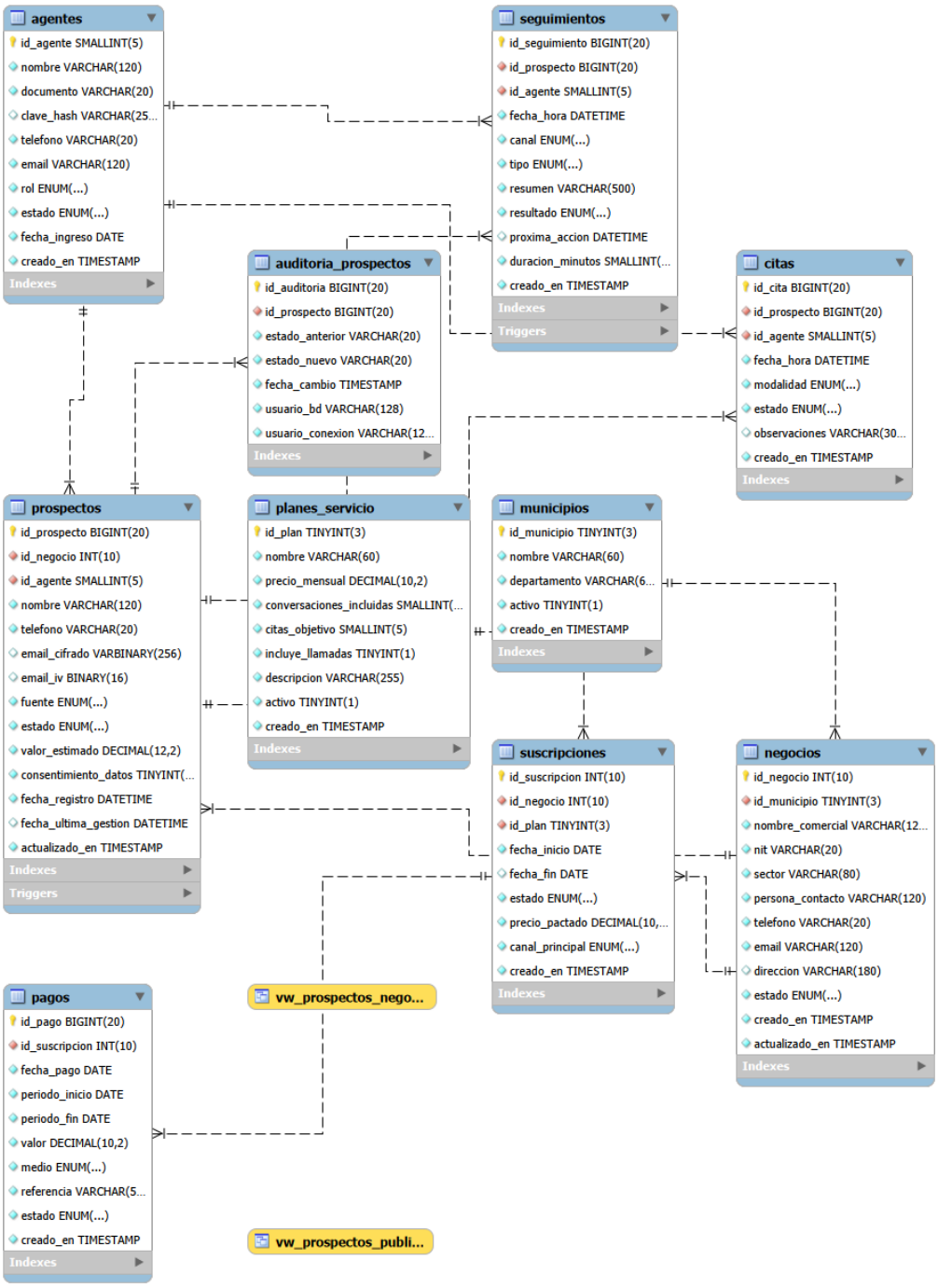


Tabla 1

Tablas y propósito

Tabla	Propósito
municipios	Ubicación de los negocios.
negocios	Empresas que reciben la atención comercial.
agentes	Responsables de atención y hash de aplicación.
prospectos	Interesados, estado, correo cifrado e IV.
seguimientos	Historial de contactos e intentos.
citas	Agenda, modalidad y estado de las citas.
planes_servicio	Catálogo del servicio ofrecido.
suscripciones	Plan y precio contratados por negocio.
pagos	Abonos y períodos de una suscripción.
auditoria_prospectos	Cambios de estado y cuentas involucradas.

Se añade agentes.clave_hash. En prospectos se sustituye email por email_cifrado y email_iv. En auditoria_prospectos se añade usuario_conexion, conservando usuario_bd. Las claves primarias y foráneas del primer corte se mantienen.

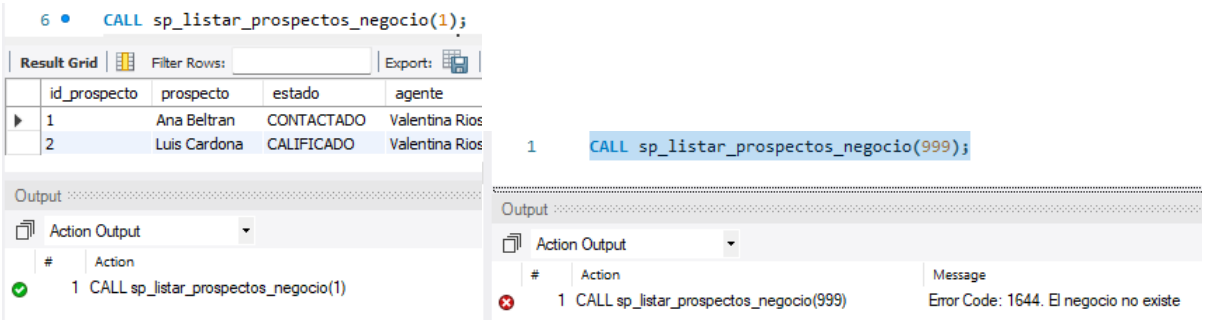
3 Procedimientos almacenados

3.1 Listar prospectos de un negocio

sp_listar_prospectos_negocio recibe IN p_id_negocio de tipo INT UNSIGNED. Comprueba que el negocio exista y devuelve ID, nombre, estado y agente de sus prospectos. El INNER JOIN encuentra al agente por su identificador; WHERE limita el resultado al negocio solicitado.

```
SELECT COUNT(*) INTO v_existe
FROM negocios WHERE id_negocio = p_id_negocio;
IF v_existe = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'El negocio no existe';
END IF;
```

La entrada 1 consulta OdontoCentro Cartago. La entrada 999 prueba la validación y produce el mensaje personalizado. Un negocio existente sin prospectos puede devolver una lista vacía; no se confunde con un negocio inexistente. SIGNAL permite detener la rutina con un mensaje definido para ese caso (MariaDB, s. f.-f).

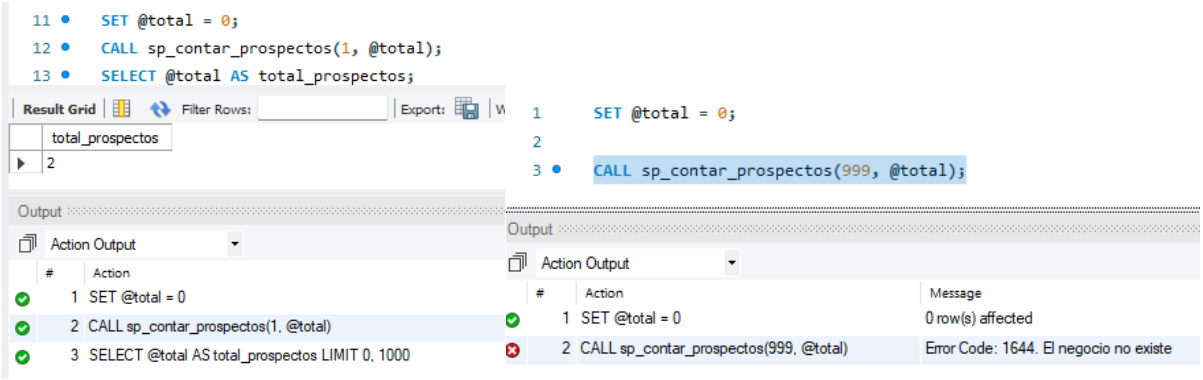


3.2 Contar prospectos de un negocio

sp_contar_prospectos recibe IN p_id_negocio y devuelve OUT p_total, de tipo INT. Valida que exista el negocio y usa COUNT(*) para calcular cuántas personas tiene registradas.

```
SELECT COUNT(*) INTO p_total
FROM prospectos
WHERE id_negocio = p_id_negocio;
```

La ejecución con negocio 1 devuelve 2 sobre los datos iniciales. La prueba con negocio 999 debe rechazarse con SIGNAL. El OUT representa un total calculado; el valor inicial del formulario de Workbench se sustituye al finalizar.



3 Procedimientos almacenados continuación

3.3 Registrar un intento de seguimiento

sp_registrar_intento comprueba las entradas y añade una interacción en seguimientos. El parámetro INOUT recibe un número de intento y devuelve ese mismo número incrementado.

Tabla 2

Parámetros de sp_registrar_intento

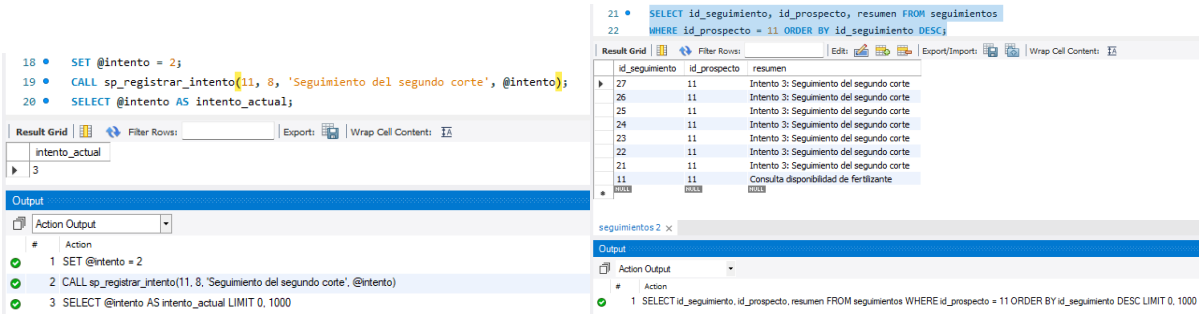
Parámetro	Modo y tipo	Uso
p_id_prospecto	IN BIGINT UNSIGNED	Persona contactada.
p_id_agente	IN SMALLINT UNSIGNED	Responsable del contacto.
p_resumen	IN VARCHAR(500)	Descripción de la gestión.
p_numero_intento	INOUT INT	Número recibido y actualizado.

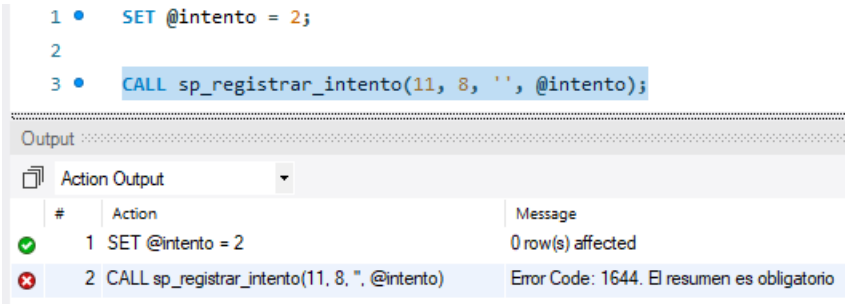
Se valida la existencia del prospecto y del agente activo, un resumen no vacío y un número de intento no negativo ni nulo. Las condiciones usan SIGNAL antes de insertar. La rutina suma uno al valor introducido; no calcula el número histórico de intentos desde la tabla.

```
SET p_numero_intento = p_numero_intento + 1;
-- Dentro del INSERT:
CONCAT('Intento ', p_numero_intento, ': ', TRIM(p_resumen))
```

La prueba utiliza prospecto 11, agente 8, resumen “Seguimiento del segundo corte” e intento 2. La respuesta es 3 y se añade una fila con ese texto. WHATSAPP es el canal registrado, SALIENTE el tipo y PENDIENTE el resultado. La fecha toma el valor por defecto de la tabla. Registrar esta interacción no envía un mensaje real.

La prueba de error utiliza un resumen vacío. El código completo está en sql/procedimientos.sql y los bloques de demostración, en sql/pruebas.sql. También puede ejecutarse con Call Stored Procedure desde Workbench.





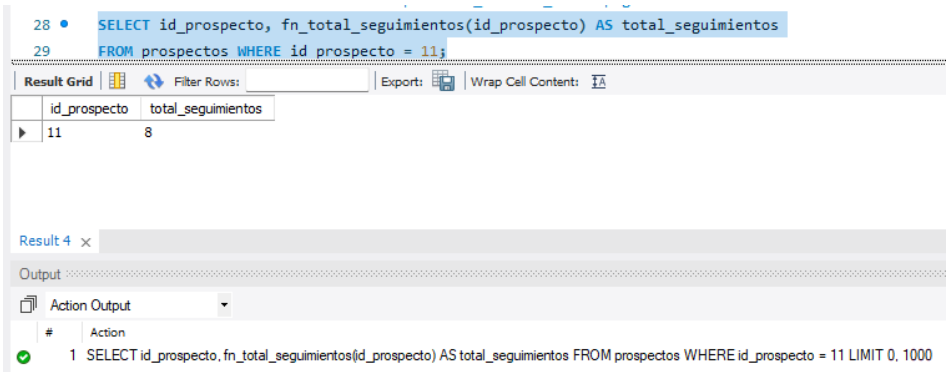
4 Funciones almacenadas

4.1 Total de seguimientos

fn_total_seguimientos recibe p_id_prospecto, de tipo BIGINT UNSIGNED, y devuelve INT. Cuenta las interacciones asociadas a esa persona y retorna el total. Se declara READS SQL DATA porque consulta tablas y NOT DETERMINISTIC porque el total puede cambiar al registrar contactos nuevos.

```
SELECT fn_total_seguimientos(11) AS total_seguimientos;
```

La función se usa dentro de un SELECT, por lo que su valor puede incorporarse a un reporte. Un prospecto sin contactos devuelve cero. El total del prospecto 11 depende de cuántas pruebas se hayan ejecutado.

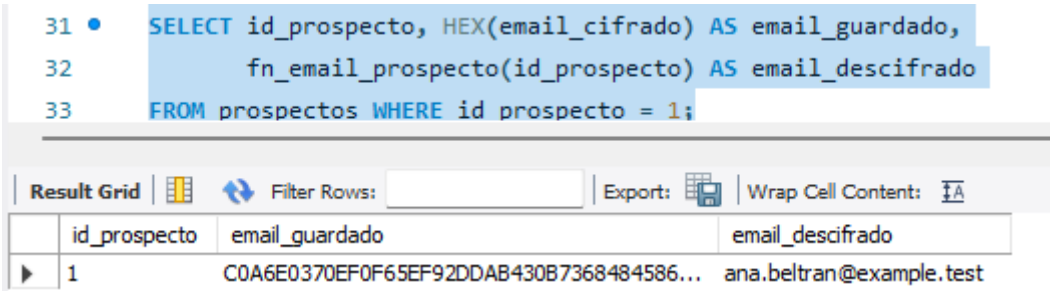


4.2 Recuperación autorizada del correo

fn_email_prospecto recibe p_id_prospecto, de tipo BIGINT UNSIGNED, y devuelve VARCHAR(120). Recupera el correo cifrado mediante AES_DECRYPT y lo convierte a texto utf8mb4. Usa la clave local y el IV guardado en la fila. Solo el rol administrador recibe permiso para ejecutarla.

```
SELECT id_prospecto, HEX(email_cifrado) AS guardado,
       fn_email_prospecto(id_prospecto) AS recuperado
FROM prospectos WHERE id_prospecto = 1;
```

La clave de 32 bytes se lee del secreto montado en el servidor y no forma parte del retorno. Si la clave no está disponible, la función lanza SIGNAL. La definición usa SQL SECURITY DEFINER para la lectura autorizada del archivo; esto no elimina la comprobación del privilegio EXECUTE de quien llama la función. MariaDB 11.4 admite especificar el IV y el modo de AES (MariaDB, s. f.-a).



5 Triggers

5.1 Auditoría de cambios de estado

trg_prospecto_auditar_estado es AFTER UPDATE sobre prospectos y actúa FOR EACH ROW. Compara OLD.estado y NEW.estado. Cuando son diferentes, inserta la transición en auditoria_prospectos; un UPDATE que conserva el estado no añade una transición (MariaDB, s. f.-c).

```
IF OLD.estado <> NEW.estado THEN
    INSERT INTO auditoria_prospectos
      (id_prospecto, estado_anterior, estado_nuevo,
       usuario_bd, usuario_conexion)
    VALUES (NEW.id_prospecto, OLD.estado, NEW.estado,
            CURRENT_USER(), USER());
END IF;
```

USER() conserva la cuenta de la conexión que originó el cambio. CURRENT_USER() dentro de este trigger corresponde a su definidor root@localhost. La fecha se agrega por DEFAULT CURRENT_TIMESTAMP. Esta distinción evita atribuir a la persona conectada la identidad del definidor (MariaDB, s. f.-e).

```
36 •  UPDATE prospectos SET estado = 'NUEVO' WHERE id_prospecto = 20;
37
38 •  UPDATE prospectos SET estado = 'CONTACTADO' WHERE id_prospecto = 20;
39 •  SELECT id_prospecto, estado_anterior, estado_nuevo, usuario_conexion, usuario_bd, fecha_cambio
40    FROM auditoria_prospectos WHERE id_prospecto = 20 ORDER BY id_auditoria DESC LIMIT 1;
41
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

id_prospecto	estado_anterior	estado_nuevo	usuario_conexion	usuario_bd	fecha_cambio
20	NUEVO	CONTACTADO	admin_nl@172.19.0.1	root@localhost	2026-10-28 20:29:03

5.2 Validación del resumen

trg_seguimiento_validar_resumen es BEFORE INSERT sobre seguimientos. Rechaza resúmenes vacíos o nulos y retira espacios exteriores con TRIM. Se aplica también a inserciones directas que no pasen por el procedimiento.

```
IF NEW.resumen IS NULL OR TRIM(NEW.resumen) = '' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'El seguimiento necesita un resumen';
END IF;
SET NEW.resumen = TRIM(NEW.resumen);
```

```
1 •  INSERT INTO seguimientos (id_prospecto, id_agente, canal, tipo, resumen, resultado)
2    VALUES (11, 8, 'WHATSAPP', 'SALIENTE', '   Contacto de prueba   ', 'PENDIENTE');
3
4 •  SELECT id_seguimiento, CONCAT('[', resumen, ']') AS resumen_guardado FROM seguimientos WHERE id_prospecto = 11 ORDER BY id_seguimiento DESC LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

id_seguimiento	resumen_guardado
28	[Contacto de prueba]

Output

Action Output

#	Action	Message
1	INSERT INTO seguimientos (id_prospecto, ...	1 row(s) affected
2	SELECT id_seguimiento, CONCAT('[', resu...	1 row(s) returned

6 Vistas

6.1 Vista operativa con JOIN

vw_prospectos_negocio reúne identificador, negocio, nombre del prospecto, estado y agente. Los dos INNER JOIN usan las claves foráneas de prospectos. La vista permite consultar la operación sin repetir nombres de negocios y agentes en el almacenamiento.

```
SELECT p.id_prospecto, n.nombre_comercial AS negocio,
       p.nombre AS prospecto, p.estado, a.nombre AS agente
FROM prospectos p
INNER JOIN negocios n ON p.id_negocio = n.id_negocio
INNER JOIN agentes a ON p.id_agente = a.id_agente;
```

El administrador y el agente pueden consultar esta vista. El rol de consulta no recibe acceso a ella, pues muestra nombres completos.

```
50 •  SELECT * FROM vw_prospectos_negocio WHERE id_prospecto <= 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_prospecto	negocio	prospecto	estado	agente
1	OdontoCentro Cartago	Ana Beltran	CONTACTADO	Valentina Rios
2	OdontoCentro Cartago	Luis Cardona	CALIFICADO	Valentina Rios

6.2 Vista con enmascaramiento

vw_prospectos_publico conserva el ID y el estado, muestra solamente la inicial del nombre y las cuatro últimas cifras del teléfono. Excluye el correo. Esta información se muestra en el panel PHP y se concede al usuario de consulta.

```
SELECT id_prospecto,
       CONCAT(LEFT(nombre,1), '***') AS nombre_visible,
       CONCAT('******',RIGHT(telefono,4)) AS telefono_visible,
       estado
FROM prospectos;
```

El enmascaramiento modifica la presentación de los datos, mientras que el cifrado modifica lo que se almacena. Para que la vista reduzca la exposición, consulta_nl no tiene SELECT sobre la tabla original. Las vistas se ejecutan con los permisos de su definidor, y el acceso de lectura se concede sobre cada vista autorizada (MariaDB, s. f.-d).

```
53 •  SELECT * FROM vw_prospectos_publico WHERE id_prospecto <= 2;
54
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_prospecto	nombre_visible	telefono_visible	estado
1	A***	******0001	CONTACTADO
2	L***	******0002	CALIFICADO

7 Seguridad

7.1 Roles usuarios y privilegios

Los permisos se agrupan en tres roles y se asignan a tres cuentas. SET DEFAULT ROLE activa el rol correspondiente al iniciar sesión. La administración de datos del proyecto queda separada de la administración global del servidor (MariaDB, s. f.-b).

Tabla 3

Matriz de seguridad por rol objeto y privilegio

Objeto	rol_admin_nl	rol_agente_nl	rol_consulta_nl
Tablas de norte_leads	SELECT, INSERT, UPDATE, DELETE	Según filas siguientes	Sin acceso directo
prospectos	Gestión de datos	SELECT (id, estado); UPDATE (estado)	Sin acceso
seguimientos	Gestión de datos	SELECT; inserta mediante rutina	Sin acceso
auditoria_prospectos	Gestión de datos	SELECT	Sin acceso
vw_prospectos_negocio	SELECT	SELECT	Sin acceso
vw_prospectos_publico	SELECT	SELECT	SELECT
Tres procedimientos	EXECUTE	EXECUTE	Sin acceso
fn_total_seguimientos	EXECUTE	EXECUTE	Sin acceso
fn_email_prospecto	EXECUTE	Sin acceso	Sin acceso
mysql.user y FILE	Sin acceso	Sin acceso	Sin acceso

Las cuentas son admin_nl, agente_nl y consulta_nl. La lectura de la clave de cifrado se realiza dentro de una función definida por root y autorizada solo al administrador, no mediante un privilegio FILE concedido a las cuentas del proyecto.

```
GRANT SELECT ON norte_leads.vw_prospectos_publico
TO rol_consulta_nl;
GRANT rol_consulta_nl TO 'consulta_nl'@'%';
SET DEFAULT ROLE rol_consulta_nl FOR 'consulta_nl'@'%';
```

El script de seguridad bloquea la cuenta amplia norteleads_app del primer corte si existe. PHP utiliza ahora consulta_nl para leer únicamente la vista enmascarada. La matriz expresa permisos del laboratorio; la separación de datos por negocio queda para la fase final.

7 Seguridad continuación

7.2 Hash de contraseñas con sal

agentes.clave_hash contiene hashes bcrypt de contraseñas de aplicación ficticias. La sal aleatoria forma parte de la cadena almacenada. Para generar y comprobar hashes se utilizan password_hash y password_verify de PHP. Dos llamadas con la misma contraseña generan cadenas distintas por la sal, y la verificación acepta la correcta y rechaza otra (PHP Documentation Group, s. f.).

```
$hash = password_hash($clave, PASSWORD_BCRYPT, ['cost' => 12]);
$correcta = password_verify($clave, $hash);
```

La demostración está en scripts/hash_demo.php y se ejecuta por terminal. No constituye todavía un login de agentes. Estas contraseñas de aplicación se distinguen de las cuentas MariaDB, cuya autenticación gestiona el motor. El hash no se descifra para comprobar una clave.

```
56 •  SELECT id_agente, clave_hash FROM agentes WHERE id_agente IN (1,2);
57
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wr

id_agente	clave_hash
1	$2b$12$d/ciNNhjvggmzmeSwdj6fOSyaYA1IfTY…
2	$2b$12$Msqa4/PTFfKaRTDbYyvqJesG4QaG4GT…
NULL	NULL

7.3

Cifrado de una columna sensible

El correo de prospectos se guarda en email_cifrado mediante AES-256-CBC. Cada fila tiene un IV de 16 bytes; la clave aleatoria local no se publica y se monta solo en MariaDB. Se elimina la columna original email para no conservar una copia legible en la tabla.

```
AES_ENCRYPT(correo, clave, iv, 'aes-256-cbc')
AES_DECRYPT(email_cifrado, clave, email_iv, 'aes-256-cbc')
```

El usuario administrador puede recuperar el correo con fn_email_prospecto. Los demás no reciben ese permiso. El secreto debe conservarse junto al respaldo, en almacenamiento separado del repositorio: sin la clave no se recuperan los correos. Esta protección se limita a esa columna y no equivale a cifrar todo el servidor ni a establecer un canal TLS.

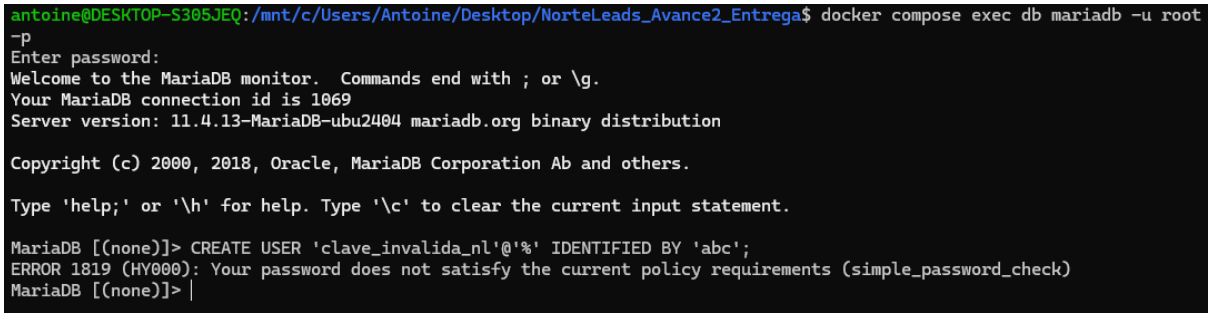
La migración modifica las columnas y conserva los registros del primer corte. Los hashes se generan con sal distinta por agente y los correos se cifran con un IV por registro. La clave no está embebida en la definición pública de una vista o función.

7 Seguridad pruebas y política

7.4 Política de contraseñas

La configuración carga simple_password_check y exige doce caracteres, al menos un número, una mayúscula, una minúscula y un símbolo. strict_password_validation permanece activo. La regla aplica a contraseñas nuevas o modificadas; no sustituye automáticamente las antiguas (MariaDB, s. f.-g).

```
[mariadb]
plugin_load_add = simple_password_check
simple_password_check_minimal_length = 12
simple_password_check_digits = 1
simple_password_check_letters_same_case = 1
simple_password_check_other_characters = 1
```



7.5 Operaciones permitidas y denegadas

Las pruebas se ejecutan en tres conexiones de Workbench. SELECT USER(), CURRENT_ROLE() identifica cada cuenta. No se utiliza root para simular a los usuarios del proyecto.

Tabla 4

Pruebas de permisos por usuario

Cuenta	Operación permitida	Operación denegada
admin_nl	SELECT del correo mediante fn_email_prospecto.	SELECT User, Host FROM mysql.user.
agente_nl	CALL sp_listar_prospectos_negocio(1).	SELECT clave_hash FROM agentes.
consulta_nl	SELECT de vw_prospectos_publico.	SELECT nombre, telefono FROM prospectos.

Un permiso denegado debe verse como un error del motor bajo la cuenta indicada. Las pruebas completas están separadas por usuario en sql/pruebas_usuarios.sql. En el video se muestran dos usuarios; en el informe deben quedar documentadas las tres cuentas.

```sql
SELECT USER(), CURRENT_USER(), CURRENT_ROLE();

SELECT
    id_prospecto,
    norte_leads.fn_email_prospecto(id_prospecto) AS email
FROM norte_leads.prospectos
WHERE id_prospecto = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_prospecto	email
1	ana.beltran@example.test

Result 9 | Result 10

Output

Action Output

#	Action	Message
1	SELECT USER(), CURRENT_USER(), CURRENT_ROLE() LIMIT 0, 1000	1 row(s) returned
2	SELECT id_prospecto, norte_leads.fn_email_prospecto(id_prospecto) AS email FROM norte_leads.prospe...	1 row(s) returned

```sql
SELECT USER(), CURRENT_USER(), CURRENT_ROLE();

SELECT User, Host
FROM mysql.user;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

USER()	CURRENT_USER()	CURRENT_ROLE()
admin_nl@172.19.0.1	admin_nl@%	rol_admin_nl

Output

Action Output

#	Action	Message
1	SELECT USER(), CURRENT_USER(), CURRENT_ROLE() LIMIT 0, 1000	1 row(s) returned
2	SELECT User, Host FROM mysql.user LIMIT 0, 1000	Error Code: 1142. SELECT command denied to user 'admin_nl'@'172.19.0.1' for table 'mysql'.'user'

```sql
SELECT USER(), CURRENT_USER(), CURRENT_ROLE();

CALL norte_leads.sp_listar_prospectos_negocio(1);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_prospecto	prospecto	estado	agente
1	Ana Beltran	CONTACTADO	Valentina Rios
2	Luis Cardona	CALIFICADO	Valentina Rios

Output

Action Output

#	Action	Message
1	SELECT USER(), CURRENT_USER(), CURRENT_ROLE() LIMIT 0, 1000	1 row(s) returned
2	CALL norte_leads.sp_listar_prospectos_negocio(1)	2 row(s) returned

```sql
SELECT USER(), CURRENT_USER(), CURRENT_ROLE();

SELECT clave_hash
FROM norte_leads.agentes;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

USER()	CURRENT_USER()	CURRENT_ROLE()
agente_nl@172.19.0.1	agente_nl@%	rol_agente_nl

Output

Action Output

#	Action	Message
1	SELECT USER(), CURRENT_USER(), CURRENT_ROLE() LIMIT 0, 1000	1 row(s) returned
2	SELECT clave_hash FROM norte_leads.agentes LIMIT 0, 1000	Error Code: 1142. SELECT command denied to user 'agente_nl'@'172.19.0.1' for table 'norte_leads'.'agentes'

```
1  SELECT USER(), CURRENT_USER(), CURRENT_ROLE();
2
3  SELECT *
4  FROM norte_leads.vw_prospectos_publico
5  WHERE id_prospecto <= 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_prospecto	nombre_visible	telefono_visible	estado
1	A***	******0001	CONTACTADO
2	L***	******0002	CALIFICADO

Output

Action Output

#	Action	Message
1	SELECT USER(), CURRENT_USER(), CURRENT_ROLE() LIMIT 0, 1000	1 row(s) returned
2	SELECT * FROM norte_leads.vw_prospectos_publico WHERE id_prospecto <= 2 LIMIT 0, 1000	2 row(s) returned

```
1  SELECT USER(), CURRENT_USER(), CURRENT_ROLE();
2
3  SELECT nombre, telefono
4  FROM norte_leads.prospectos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

USER()	CURRENT_USER()	CURRENT_ROLE()
consulta_rl@172.19.0.1	consulta_rl@%	rol_consulta_rl

Output

Action Output

#	Action	Message
1	SELECT USER(), CURRENT_USER(), CURRENT_ROLE() LIMIT 0, 1000	1 row(s) returned
2	SELECT nombre, telefono FROM norte_leads.prospectos LIMIT 0, 1000	Error Code: 1142. SELECT command denied to user 'consulta_rl'@'172.19.0.1' for table 'norte_leads'.'prospect

8 Dificultades y próximos pasos

El repositorio del primer avance contenía referencias de Compose a carpetas app y sql/init que no estaban publicadas. Se corrigieron el contexto de construcción y los montajes para utilizar los archivos de la entrega. También se añadió index.php, necesario para construir la imagen de la aplicación.

Otra dificultad fue actualizar el esquema con información existente. Se incorporó un respaldo previo y una migración que cifra los correos y agrega columnas sin repetir los datos de prueba. Los registros antiguos de auditoría no tienen una cuenta de conexión registrada; se identifican como NO_REGISTRADO_AVANCE_1, en lugar de atribuirles una cuenta inventada.

Fue necesario distinguir el usuario conectado de la cuenta bajo la que se ejecuta el trigger. Se conservó CURRENT_USER() en usuario_bd y se añadió USER() en usuario_conexion. La nueva auditoría permite observar ambos valores.

Para el proyecto final quedan la autenticación real de agentes en PHP, permisos limitados por negocio, manejo de nuevas altas con correo cifrado y una estrategia de recuperación de la clave. También se atenderá la retroalimentación escrita del docente y se documentará cada ajuste.

9 Uso de inteligencia artificial

Se utilizó ChatGPT con Codex para revisar los lineamientos, comparar el repositorio, proponer las validaciones y preparar explicaciones del código. El código se contrastó con documentación oficial de MariaDB y PHP, y se realizaron comprobaciones funcionales en un motor MariaDB 11.4.5. Se incluyen consultas reproducibles para verificar los permisos y obtener las capturas reales en Docker. El estudiante debe ejecutar esas pruebas en su entorno y explicar los resultados de su grabación. Las capturas pendientes y el video no se sustituyen por resultados generados.

Referencias

MariaDB. (s. f.-a). *AES_ENCRYPT*. MariaDB Documentation. https://mariadb.com/docs/server/reference/sql-functions/secondary-functions/encryption-hashing-and-compression-functions/aes_encrypt

MariaDB. (s. f.-b). *CREATE ROLE*. MariaDB Documentation. https://mariadb.com/docs/server/reference/sql-statements/account-management-sql-statements/create-role

MariaDB. (s. f.-c). *CREATE TRIGGER*. MariaDB Documentation. https://mariadb.com/docs/server/server-usage/triggers-events/triggers/create-trigger

MariaDB. (s. f.-d). *CREATE VIEW*. MariaDB Documentation. https://mariadb.com/docs/server/server-usage/views/create-view

MariaDB. (s. f.-e). *CURRENT_USER*. MariaDB Documentation. https://mariadb.com/docs/server/reference/sql-functions/secondary-functions/information-functions/current_user

MariaDB. (s. f.-f). *SIGNAL*. MariaDB Documentation. https://mariadb.com/docs/server/reference/sql-statements/programmatic-compound-statements/signal

MariaDB. (s. f.-g). *Simple Password Check Plugin*. MariaDB Documentation. https://mariadb.com/docs/server/reference/plugins/password-validation-plugins/simple-password-check-plugin

PHP Documentation Group. (s. f.). *password_hash*. Manual de PHP. https://www.php.net/manual/es/function.password-hash.php

Anexo A Evidencias de ejecución

Las siguientes figuras documentan la ejecución real de los procedimientos, funciones, triggers, vistas y controles de seguridad implementados en NorteLeads. Las capturas fueron obtenidas mediante MySQL Workbench y la terminal de WSL sobre los contenedores del proyecto.

Tabla A1

Relación de evidencias de ejecución

Figura	Archivo inicia con	Contenido
2	01	P1 ejecución válida
3	02	P1 validación con error
4	03	P2 OUT correcto
5	04	P2 validación con error
6	05	P3 INOUT y fila nueva
7	06	P3 resumen rechazado
8	07	Función total en SELECT
9	08	Correo cifrado y recuperación
10	09	Trigger de auditoría
11	10	Trigger que limpia resumen
12	11	Vista con JOIN
13	12	Vista enmascarada
14	13	Hashes almacenados
15	14	Hash y verificación en PHP
16	15	Admin permitido
17	16	Admin denegado
18	17	Agente permitido
19	18	Agente denegado
20	19	Consulta permitido
21	20	Consulta denegado
22	21	Política de contraseña